Loading Selenium WebDriver

```
public class SeleniumIntroduction {
   public static void main() {
System.setProperty("webdriver.chrome.driver","C:\\Users\\huzai\\Downloads\\chromed
river-win64\\chromedriver-win64");
      //This is manual. If we skip this then Selenium manager will download it
automatically
      WebDriver driver = new ChromeDriver();
   }
}
```

Loading Appium Drivers : Android and iOS

```
public XCUITestOptions getIOSOptions() {  1 usage   smahuzaifa
    XCUITestOptions options = new XCUITestOptions();
    options.setDeviceName("iPhone 17 Pro");
    options.setApp("/Users/huzaifa/IdeaProjects/AppiumPOM/src/test/java/resources/UIKitCa
    options.setPlatformVersion("26.1");
    options.setUdid("AB44849F-788D-480A-89A8-3A4DB94B2742");
    return options;
}
public UiAutomator2Options getAndroidOptions() {  1 usage   smahuzaifa *
    UiAutomator2Options options = new UiAutomator2Options();
    options.setDeviceName("Pixel 9 Pro"); //Emulator
    options.setApp("C:\\Users\\huzai\\IdeaProjects\\AppiumPOM\\src\\test\\java\\resources
    return options;
}
```

iOS

```
public class IOSBaseTest {
   public IOSDriver driver;
   public AppiumDriverLocalService service;
   @BeforeClass
   public void configuringAppium() throws Exception {
      String os = System.getProperty("os.name").toLowerCase();
      String appiumJSPath;

      if (os.contains("win")) {
         appiumJSPath =
"C:\\Users\\huzai\\AppData\\Roaming\\npm\\node_modules\\appium\\build\\lib\\main.
js";
      } else if (os.contains("mac")) {
         appiumJSPath = "/opt/homebrew/lib/node_modules/appium/build/lib/main.js";
      } else {
```

```
        throw new RuntimeException("Unsupported OS: " + os);
    }

    service = AppiumDriverLocalService.buildService(
        new AppiumServiceBuilder()
            .withAppiumJS(new File(appiumJSPath))
            .withIPAddress("127.0.0.1")
            .usingPort(4723)
            .withArgument(GeneralServerFlag.BASEPATH, "/wd/hub")
    );
    //service.start();
    //Android uses UiAutomator2
    XCUITestOptions options = new XCUITestOptions();
    options.setDeviceName("iPhone 17 Pro");

options.setApp("/Users/huzaifa/IdeaProjects/Appium/src/test/java/resources/UIKitCata
log.app");
    options.setPlatformVersion("26.1");
    options.setUdid("AB44849F-788D-480A-89A8-3A4DB94B2742");
    //In iOS, we have to install web driver agent and then using that app and options the
app is automated
    options.setWdaLaunchTimeout(Duration.ofSeconds(20));

    driver = new IOSDriver(new URI("http://127.0.0.1:4723").toURL(),options);
    driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
  }
```

Android

```
public class BaseTest {
  public AndroidDriver driver;
  public AppiumDriverLocalService service;
  @BeforeClass
  public void configuringAppium() throws Exception {
    String os = System.getProperty("os.name").toLowerCase();
    String appiumJSPath;

    if (os.contains("win")) {
      appiumJSPath =
"C:\\Users\\huzai\\AppData\\Roaming\\npm\\node_modules\\appium\\build\\lib\\main.
```

```
js";
    } else if (os.contains("mac")) {
      appiumJSPath = "/opt/homebrew/lib/node_modules/appium/build/lib/main.js";
    } else {
      throw new RuntimeException("Unsupported OS: " + os);
    }

    service = AppiumDriverLocalService.buildService(
        new AppiumServiceBuilder()
            .withAppiumJS(new File(appiumJSPath))
            .withIPAddress("127.0.0.1")
            .usingPort(4723)
            .withArgument(GeneralServerFlag.BASEPATH, "/wd/hub") // <-- important
    );
    service.start();

    UiAutomator2Options options = new UiAutomator2Options();
    options.setDeviceName("Pixel 9 Pro");
    String apkPath;
    if (os.contains("win")) {
      apkPath =
"C:\\Users\\huzai\\IdeaProjects\\Appium\\src\\test\\java\\resources\\ApiDemos-
debug.apk";
    } else if (os.contains("mac")) {
      apkPath =
"/Users/huzaifa/IdeaProjects/Appium/src/test/java/resources/ApiDemos-debug.apk";
    } else {
      throw new RuntimeException("Unsupported OS: " + os);
    }
    options.setApp(apkPath);


    //Creating androidDriver object
    driver = new AndroidDriver(new URI("http://127.0.0.1:4723").toURL(),options);
    // Uniform Resource Identifier (URI) is a string that identifies a resource, while a
    // Uniform Resource Locator (URL) is a type of URI that specifies both the identity
and the
    // location of a resource, typically on the web

    //Adding timeouts or waits
    driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

```
    //this will wait a maximum of 10 seconds for each execution or line of code
  }
```

Configuring Appium

```
public class AppiumUtils {
  protected AppiumDriver driver;
  public AppiumDriverLocalService service;//Parent Driver which works with both
android and iOS
  //This is grandparent which is shared between both android and iOS actions
  public void init(AppiumDriver driver) {
    this.driver = driver;
    PageFactory.initElements(new AppiumFieldDecorator(driver), this);
  }

  public AppiumDriverLocalService startingAppium() throws IOException {
    //To Start the Appium Server automatically
    String os = System.getProperty("os.name").toLowerCase();
    String appiumJSPath;
    Properties prop = new Properties();
    FileInputStream fis = new
FileInputStream(System.getProperty("user.dir")+"/src/test/java/resources/data.properti
es");
    prop.load(fis);
    String ip = prop.getProperty("ipAddress");
    int port = Integer.parseInt(prop.getProperty("port"));
    System.out.println(System.getProperty("user.dir"));
    System.out.println("The IP is "+ip +" and the port is "+port);
    if (os.contains("win")) {
      appiumJSPath =
"C:\\Users\\huzai\\AppData\\Roaming\\npm\\node_modules\\appium\\build\\lib\\main.
js";
    } else if (os.contains("mac")) {
      appiumJSPath = "/opt/homebrew/lib/node_modules/appium/build/lib/main.js";
    } else {
      throw new RuntimeException("Unsupported OS: " + os);
    }
    service = AppiumDriverLocalService.buildService(
        new AppiumServiceBuilder()
```

```
            .withAppiumJS(new File(appiumJSPath))
            .withIPAddress(ip)
            .usingPort(port)
            .withArgument(GeneralServerFlag.BASEPATH, "/wd/hub")
    );
    return service;
}
```